

Invoice chaser for freelancers

Web app for solo designers and developers invoicing 3–10 clients a month. Scoped to 8 weeks and a low-code build.

01 Problem and target user

Solo designers and developers invoice a handful of clients a month and get paid late on a predictable share of them. Chasing is a social cost, not a technical one: the first reminder is easy, the second feels like begging, and the third never gets sent. The result is unpaid work carried for weeks and, on a small number of invoices, written off entirely.

PRIMARY USER

- Freelancer billing 3–10 clients a month, invoice values \$500–\$8,000.
- Already sends invoices from Stripe, Wave or a spreadsheet — will not migrate their invoicing.
- Comfortable in web tools, not interested in configuring automations.

EXPLICITLY NOT THE USER

- Agencies with a bookkeeper, who need approvals and multi-seat permissions.
- Businesses needing legal collections or debt sale.

02 Core features

The MVP does one job end to end: know what is owed, and send the next reminder without the user writing it. Anything that does not serve that loop is deferred.

MUST HAVE

- Invoice import — Stripe connection plus CSV upload, with amount, client, due date.
- Reminder schedule — a default cadence (3 days before, day of, +7, +21) editable per invoice.
- Message templates — three tones, editable, with merge fields for name, amount, due date.
- Status board — every invoice as upcoming, due, overdue or paid, sorted by money at risk.
- Send and log — email sends from the user's address, and every send is visible on the invoice.

SHOULD HAVE, POST-LAUNCH

- Client-facing status page with a pay link.
- Weekly Monday digest: what is owed, what goes out this week.
- Partial payment tracking.

WILL NOT HAVE IN THE MVP

- Invoice creation or editing — this is not an invoicing tool.
- SMS or WhatsApp channels.
- Multi-user accounts, roles, or client portals with logins.
- Accounting integrations beyond Stripe.

03 User stories

Written from the freelancer's side. Each is testable in the built MVP.

SETUP

- As a freelancer, I connect Stripe once and see my open invoices without re-entering them.
- As a freelancer, I upload a CSV when a client pays me outside Stripe.
- As a freelancer, I pick a reminder cadence once and have it apply to every new invoice.

DAY TO DAY

- As a freelancer, I open the board and see the single largest overdue invoice first.
- As a freelancer, I approve a queued reminder in one click, or edit the wording before it goes.
- As a freelancer, I pause reminders on an invoice when a client tells me payment is coming.
- As a freelancer, I see that a reminder was sent and opened, so I do not chase twice.

04 User flows

Three flows carry the product. The first-run flow must end with at least one real invoice on the board, or the user has no reason to return.

FIRST RUN — SIGNUP TO FIRST INVOICE

- Sign up with email!' connect Stripe or upload CSV!' confirm the imported invoices land on the board with reminders queued.
- Fallback: no invoices to import. Offer manual entry of one invoice so the board is never empty.

REMINDER APPROVAL

- Digest email or board badge!' queued reminders view!' read the draft!' send the invoice timeline.

INVOICE PAID

- Stripe webhook marks paid!' invoice moves to paid, remaining reminders confirmed on the board.
- Manual case: user marks paid themselves, with the same cancellation behaviour.

05 Tech stack

Chosen for one developer at a moderate comfort level shipping in eight weeks. Managed services over self-hosted, and no infrastructure the user has to think about.

BUILD

- Frontend: Next.js with TypeScript and Tailwind — one deployable, server actions for the small amount of backend work.
- Backend and data: Supabase (Postgres, auth, row-level security). Tables: user, invoice, reminder, send_log.
- Scheduling: a single cron job hitting one endpoint that queues the day's reminders. No queue system in the MVP.
- Email: Resend, with domain authentication so reminders send from the freelancer's address.
- Payments data: Stripe API for invoice read and paid webhooks.
- Hosting: Vercel. Error tracking with Sentry from day one.

DELIBERATELY DEFERRED

- No background worker infrastructure, no Redis, no custom auth.

06 Risks and assumptions

The two risks that can end the product are deliverability and trust. Both need a check inside the eight weeks, not after.

RISKS

- Deliverability: reminders landing in spam makes the product invisible. Mitigation — require domain verification at setup and show a send-health state.
- Trust in automation: users may not let the tool send unsupervised. Mitigation — approval-required by default, auto-send as an opt-in after the first month.
- Stripe-only import limits the reachable market. Mitigation — CSV covers the rest at launch; measure how many use it.
- Tone: a badly worded reminder damages a client relationship. Mitigation — conservative default copy, always previewable.

ASSUMPTIONS TO VALIDATE

- Freelancers will connect a payment account to a new tool in the first session.
- Late payment is frequent enough to justify a recurring subscription, not a one-off.
- A reminder from the freelancer's own address performs better than one from the product.